MIT-BIH Arrhythmia Classification with Five Temporal Models

Zeyuan Li
Course Assignment Report

Abstract

This report summarizes a five-class heartbeat classification assignment on the MIT-BIH Arrhythmia Database. Each input sample is a 300-point single-lead ECG window centered around one heartbeat, and the goal is to classify it into one of five classes: normal beat (N), atrial premature beat (A), ventricular premature beat (V), left bundle branch block (L), and right bundle branch block (R). I implemented and compared five temporal models: LightCNN, LSTM, NM2019, Transformer, and TransformerCNN. Experiments were performed with stratified k-fold cross validation under five random seeds (0 4). NM2019 achieved the best average accuracy, $98.33 \pm 0.59\%$, while TransformerCNN and LSTM also produced competitive results above 97.9%. Complementary analyses using complexity visualization, UMAP feature visualization, and class-wise temporal saliency were used to compare accuracy, efficiency, representation quality, and interpretability. The codes are available at **https://github.com/Lucien57/MIT-BIH-decoding**

1 Task and Dataset

The assignment is a five-class arrhythmia classification task based on MIT-BIH heartbeat windows. The original ECG signals were sampled at 360 Hz, and the provided input has already been preprocessed by denoising and baseline-drift correction. Each sample is a length-300 time series corresponding to one heartbeat cycle, roughly covering the region from before the R peak to after the main beat morphology. The five labels are N, A, V, L, and R. Therefore, the task is a compact but meaningful time-series classification problem: the model must distinguish local waveform morphology, class-dependent rhythm features, and subtle changes around the QRS complex and neighboring intervals.

The main evaluation target of the assignment is to reach at least 95% test accuracy. Since a single random split may be unstable for this imbalanced dataset, I used stratified k-fold cross validation and repeated the experiment under seeds 0, 1, 2, 3, and 4. The final comparison is reported as the mean and standard deviation of accuracy across seeds.

2 Implemented Models

The five models are designed to cover common temporal modeling choices: convolution, recurrence, residual convolution, self-attention, and CNN-Transformer hybrid modeling. The implementation keeps the input format consistent across all models: a single-channel 300-point ECG segment is mapped to a five-class prediction by a feature extractor and a classification head. Table 1 summarizes the actual configuration choices used in the experiments.

2.1 LightCNN

LightCNN is a compact one-dimensional convolutional baseline. It consists of four depthwise-separable residual blocks. The channel widths are $(32, 64, 96, 128)$, the temporal kernels are $(9, 7, 5, 3)$, and the strides are $(1, 2, 2, 2)$. Each block first applies a depthwise temporal convolution, then a pointwise convolution, batch normalization, GELU activation, and dropout. When the temporal resolution or channel number changes, a 1×1 shortcut is used to keep the residual addition valid.

After the encoder, the model applies both adaptive average pooling and adaptive max pooling. The two pooled vectors are concatenated before the final linear classifier. This design makes LightCNN cheap but still expressive enough to learn local ECG morphology. Its weakness is that it compresses the signal aggressively and has limited capacity, so it may miss subtle abnormal patterns in minority classes.

2.2 LSTM

The LSTM model treats the ECG window as a sequence of 300 time steps. Since each time step contains only one input feature, a linear projection first maps the scalar value to a 32-dimensional embedding, followed by layer normalization and GELU activation. The projected sequence is processed by a two-layer bidirectional LSTM with hidden size 96 in each direction. Therefore, each time step after recurrence has a 192-dimensional representation.

For classification, I used the *last-mean* pooling option: the feature is the average of the final hidden state and the temporal mean hidden state. This is safer than using only the last step, because the heartbeat window is centered around an R peak and useful morphology may appear before or after the final sample. The pooled feature is then passed through layer normalization, dropout, and a linear head.

2.3 NM2019

NM2019 is adapted from the residual 1-D CNN proposed by Hannun et al. for cardiologist-level arrhythmia detection in ambulatory ECG recordings [1]. The implementation also refers to the open-source ECG MIT-BIH implementation at [2]. The original method was designed for long ECG records and dense sequence-level prediction. For this assignment, I kept the residual temporal convolution backbone but changed the output head to global temporal pooling followed by a single linear classifier, so that the model predicts one class for each

300-point heartbeat window.

The configured NM2019 model starts with 32 filters and uses a stack of 16 pre-activation residual blocks. Each block contains two same-padded 1-D convolutions with kernel length 16. The subsampling pattern is $[1, 2]$ repeated eight times, so the network alternates between feature refinement and temporal downsampling. The number of filters doubles every four residual blocks, producing channel stages of 32, 64, 128, and 256. The shortcut path uses max pooling and zero-padding when resolution or channel number changes. This is the largest model in this report, but it has the strongest residual temporal hierarchy and the widest receptive field.

2.4 Transformer

The Transformer model first converts the 300-point ECG segment into local temporal tokens. It uses a convolutional patch embedding with model dimension 64, kernel size 6, and stride 3. With same-style padding, the input is converted into about 101 tokens. This tokenization is more stable than feeding all raw samples as independent tokens, because each token already summarizes a short local waveform segment.

Sinusoidal positional encoding is added to the token sequence. The encoder contains three Transformer encoder layers with four attention heads, feed-forward dimension 128, GELU activation, dropout 0.1, and pre-layer normalization. The final representation is obtained by mean pooling over all tokens. This architecture can directly connect distant temporal regions, but it has weaker built-in local waveform bias than CNNs.

2.5 TransformerCNN

TransformerCNN is a parallel hybrid model. One branch is a CNN branch for local morphology and robust short-range features; the other branch is a Transformer branch for longer-range temporal interactions. The CNN branch has channels $(24, 48, 64)$, kernels $(9, 7, 5)$, and strides $(1, 2, 2)$, followed by adaptive average and max pooling. The Transformer branch uses convolutional patch embedding with $d = 48$, kernel size 6, stride 3, two encoder layers, four heads, feed-forward dimension 96, and mean pooling. The two branches produce a 176-dimensional fused feature, which is passed through layer normalization and a small MLP classifier.

This model is important because it tests whether a small hybrid design can approach the accuracy of a much deeper residual CNN. Its results suggest that this is largely possible: the CNN branch supplies useful local inductive bias, while the Transformer branch adds flexible temporal interaction without requiring a huge parameter budget.

3 Experimental Protocol

The training pipeline uses cross-entropy loss for the five-class classification task. The data are normalized by sample-wise z-score normalization, and augmentation is enabled in the logged runs. Because the class distribution is highly imbalanced, balanced class weights are passed to the cross-entropy loss. In the representative fold, the total class counts are $[71723, 1950, 6974, 6578, 4967]$ for N, A, V, L, and R, and the corresponding training class weights are approximately $[0.0681, 2.5052, 0.7006, 0.7424, 0.9837]$. This prevents the model from optimizing only for the dominant N class.

For each run, the dataset is split by stratified 10-fold cross validation. In each fold, one fold is used as the test set and another fold is used as the validation set, while the remaining folds are used for training. The validation set is used for model selection: the checkpoint with the lowest validation loss is saved as the best model and then evaluated on the test set. The implementation records standard classification metrics, including accuracy, balanced accuracy, macro-F1, weighted-F1, macro precision, macro recall, and, when applicable, one-vs-rest AUC. It also saves the test confusion matrix and the full classification report for each run. This is important for MIT-BIH because accuracy alone can hide minority-class errors.

3.1 Metric Definitions

Let $C = 5$ be the number of classes, N be the total number of test samples, and n_c be the number of samples from class c. For class c, define TP_c, FP_c, and FN_c as the numbers of true positives, false positives, and false negatives. The overall accuracy is

$$\mathrm{Acc} = \frac{\sum_{c=1}^{C} TP_c}{N}. \tag{1}$$

Class-wise precision, recall, and F1-score are defined as

$$P_c = \frac{TP_c}{TP_c + FP_c}, \quad R_c = \frac{TP_c}{TP_c + FN_c}, \quad F1_c = \frac{2P_cR_c}{P_c + R_c}. \tag{2}$$

The reported macro scores average all classes equally,

$$\begin{aligned} \text{Macro-Prec} &= \frac{1}{C}\sum_{c=1}^{C} P_c, \\ \text{Macro-Rec} &= \frac{1}{C}\sum_{c=1}^{C} R_c, \\ \text{Macro-F1} &= \frac{1}{C}\sum_{c=1}^{C} F1_c. \end{aligned} \tag{3}$$

Balanced accuracy is equivalent to macro recall in this multi-class setting. Weighted-F1 uses class support as the weight,

$$\text{Weighted-F1} = \sum_{c=1}^{C} \frac{n_c}{N} F1_c. \tag{4}$$

For AUC, each class is treated as a one-vs-rest binary problem, and the class-wise AUCs are averaged either uniformly (macro AUC) or by class support (weighted AUC). These metrics are necessary because the normal beat class is much larger than the abnormal classes; a high accuracy may still hide poor minority-class precision or recall.

Table 2 shows that all models reach the 95% target, while LightCNN only barely passes it and the other four models clearly exceed it. NM2019 obtains the best mean accuracy of 98.33%. This is consistent with its stronger residual convolutional backbone and larger temporal receptive field. For

Table 1: Implementation details of the five configured models. The parameter counts and training settings are taken from the saved logs.

Model	Main architecture configuration	Pooling / classifier	Training setting
LightCNN	4 separable residual Conv1D blocks; channels $(32, 64, 96, 128)$; kernels $(9, 7, 5, 3)$; strides $(1, 2, 2, 2)$; block dropout 0.15	Adaptive avg+max pooling; dropout 0.25; linear head	44,430 params; batch 256; LR 10^{-3}; WD 10^{-4}
LSTM	Input projection $1 \rightarrow 32$; 2-layer BiLSTM; hidden size 96 per direction; dropout 0.2	$0.5\times$(last hidden + mean hidden); LayerNorm; linear head	324,037 params; batch 128; LR 10^{-3}; WD 5×10^{-5}
NM2019	Stem Conv1D with 32 filters; 16 residual blocks; kernel 16; subsampling $[1, 2] \times 8$; filters double every 4 blocks; dropout 0.2	Global average pooling; linear head	10,462,597 params; batch 128; LR 10^{-3}; WD 10^{-4}
Transformer	Conv patch embedding; $d = 64$; patch size 6; stride 3; 3 encoder layers; 4 heads; FFN dim 128; dropout 0.1	Mean token pooling; dropout; linear head	101,253 params; batch 128; LR 5×10^{-4}; WD 10^{-4}
TransformerCNN	CNN branch $(24, 48, 64)$ with kernels $(9, 7, 5)$; Transformer branch $d = 48$, patch size 6, stride 3, 2 layers, 4 heads, FFN dim 96	Concatenated CNN+Transformer feature; LayerNorm; MLP $176 \rightarrow 128 \rightarrow 5$	85,869 params; batch 128; LR 7×10^{-4}; WD 10^{-4}

Table 2: Accuracy (%) of the five implemented models under seeds 0–4. The final column reports mean $\pm$ standard deviation across seeds.

Model	Seed 0	Seed 1	Seed 2	Seed 3	Seed 4	Average
LightCNN	94.63	95.35	95.64	95.21	94.72	95.11 ± 0.43
LSTM	98.20	98.77	97.15	98.05	97.67	97.97 ± 0.60
NM2019	99.05	98.05	98.90	97.79	97.88	$\mathbf{98.33 \pm 0.59}$
Transformer	98.47	96.91	97.13	97.73	97.51	97.55 ± 0.61
TransformerCNN	97.52	98.41	98.18	98.34	97.67	98.02 ± 0.40

ECG heartbeat classification, this advantage is intuitive: the model should not only detect the existence of a sharp QRS complex, but also compare its width, slope, and surrounding waveform pattern across a short temporal interval. A deeper residual CNN can gradually enlarge the receptive field while keeping local morphology stable through convolutional filters.

TransformerCNN is the second-best model with $98.02 \pm 0.40\%$, suggesting that combining local CNN features with Transformer-based global interaction is effective for this task. It also has the smallest standard deviation among the high-performing models, which indicates relatively stable behavior across random seeds. LSTM is also competitive, reaching 97.97%. This result confirms that recurrent sequence modeling is still a solid choice for short ECG windows, although its sequential computation is less efficient than convolution or attention. The pure Transformer performs well but is slightly weaker than TransformerCNN, probably because convolutional inductive bias remains valuable for short, morphology-driven ECG segments. In other words, attention is useful, but it should not fully replace the local filters that naturally match ECG waveform patterns. LightCNN is the weakest in accuracy, but it is also the smallest and fastest model. Therefore, it should not be treated as a failed model; rather, it is a useful lightweight deployment baseline when computation is very limited.

The detailed metrics in Table 3 also explain why I did not rely only on accuracy. The largest gap appears for LightCNN: its representative accuracy is 95.21%, but its macro-F1 is only 87.96%. This means that the lightweight model handles the abundant N class reasonably well, but its precision on minority abnormal classes is weaker. The stronger models reduce this gap. LSTM, NM2019, Transformer, and TransformerCNN all reach macro-F1 above 92% in the selected representative runs. NM2019 has the best five-seed mean accuracy, while TransformerCNN remains close with far fewer parameters and a strong balanced-accuracy score. In an imbalanced medical time-series task, this distinction matters: weighted metrics mainly reflect performance on frequent classes, while macro metrics punish minority-class errors more directly.

Two representative prediction tables are shown in Tables 4 and 5. Rows denote true labels and columns denote predicted labels in the order N, A, V, L, and R. The diagonal cells are highlighted to make correct predictions visually clear.

Most mistakes occur between N and A, which is reasonable because atrial premature beats may have relatively subtle morphology changes in a short heartbeat window. In contrast, V, L, and R are nearly perfectly separated in these representative runs, reflecting their more distinctive waveform patterns. The confusion tables also show why macro metrics are useful: even a small number of extra errors in A can noticeably affect the macro-F1 because A has much smaller support than N.

4 Complementary Experiments

The main accuracy table only gives the final predictive performance. To understand the behavior of the models more deeply, I also performed three complementary analyses: complexity visualization, UMAP feature visualization, and class-wise temporal saliency.

4.1 Complexity Visualization

Figure 1 compares accuracy, MACs, and parameter counts. The x-axis represents MACs, the y-axis represents accuracy, and the bubble annotation shows the number of trainable parameters. NM2019 has the highest accuracy, but it is much larger, with about 10.46M parameters. In contrast, LightCNN has only about 44.43K parameters and very low computation, but its accuracy is also lower. This forms a clear accuracy-

Table 3: Representative detailed test results from the saved logs. One run is selected for each model, using the seed whose accuracy is closest to the model's five-seed mean. Values except loss are shown in percent.

Run	Best epoch	Loss	Acc.	Bal. Acc.	Macro-F1	Weighted-F1	Macro Prec.	Macro Rec.	Macro AUC
LightCNN, seed 3, fold 5	81	0.1575	95.21	95.58	87.96	95.71	83.79	95.58	99.39
LSTM, seed 3, fold 5	28	0.1191	98.05	98.17	93.59	98.22	90.77	98.17	99.82
NM2019, seed 1, fold 5	6	0.0642	98.05	98.48	94.51	98.14	91.49	98.48	99.89
Transformer, seed 3, fold 5	26	0.1118	97.73	96.86	92.83	97.88	90.01	96.86	99.74
TransformerCNN, seed 2, fold 5	32	0.0922	98.18	97.99	93.91	98.31	91.18	97.99	99.81

Table 4: Representative prediction table for NM2019, seed 0, fold 5.

True / Pred.	N	A	V	L	R
N	**7108**	46	17	1	0
A	14	**178**	2	0	1
V	4	0	**694**	0	0
L	2	0	0	**655**	0
R	0	1	0	0	**496**

Table 5: Representative prediction table for TransformerCNN, seed 1, fold 5. This run has high balanced accuracy and gives a compact example of the hybrid model's prediction pattern.

True / Pred.	N	A	V	L	R
N	**7044**	84	26	14	4
A	6	**186**	1	0	2
V	5	0	**691**	2	0
L	2	0	0	**655**	0
R	0	1	0	0	**496**

efficiency trade-off: increasing model capacity improves performance, but the gain from 98.02% to 98.33% may not always justify a much larger model.

TransformerCNN is a good compromise: it achieves near-NM2019 accuracy with far fewer parameters. This suggests that a carefully designed hybrid model can be more parameter-efficient than directly using a very deep residual CNN. From a practical perspective, this matters because ECG analysis may be deployed on resource-limited devices or used in large-scale screening pipelines. In such cases, a slightly lower but more efficient model can be preferable to the absolute best model. Therefore, NM2019 is the best choice when accuracy is prioritized, while TransformerCNN is the more balanced choice when both accuracy and computation are considered.

4.2 UMAP Visualization

UMAP is used to project either the raw ECG windows or the learned feature vectors before the final classifier into a two-dimensional space. Figure 2 shows the raw data and the five model feature spaces. The raw ECG space contains many mixed regions, especially around the dominant normal class. This is expected because the raw waveform is high-dimensional and includes both class-related morphology and nuisance variation such as amplitude differences and local timing shifts. Therefore, raw Euclidean similarity does not always correspond to class similarity.

UMAP algorithm. UMAP first builds a weighted nearest-neighbor graph in the original feature space. For each sample x_i, it finds its k nearest neighbors and computes a local connectivity radius ρ_i and scale σ_i. A fuzzy membership weight from i to its neighbor j is then defined as

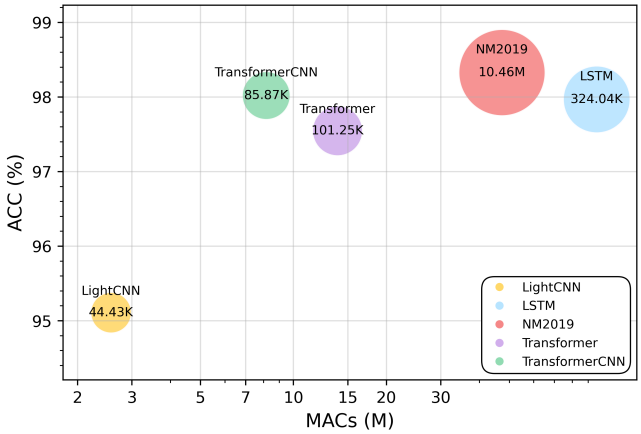


Figure 1: Accuracy-complexity comparison of the five models. Bubble annotations show trainable parameter counts. NM2019 gives the best accuracy but has the largest model size, while TransformerCNN provides a strong accuracy-efficiency trade-off.

$$v_{j|i} = \exp\left(-\frac{\max(0, d(x_i, x_j) - \rho_i)}{\sigma_i}\right). \tag{5}$$

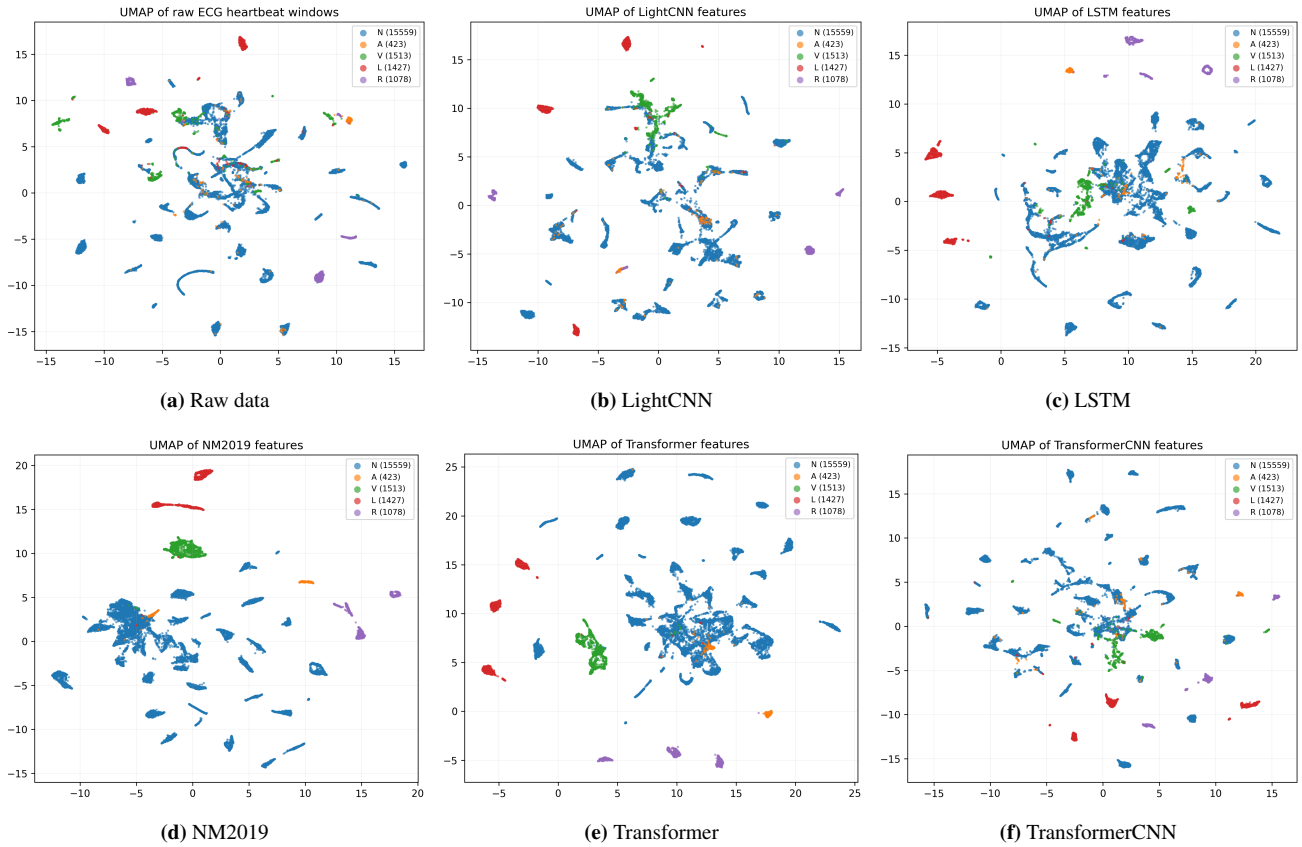
The directed memberships are symmetrized by fuzzy set union,

$$v_{ij} = v_{j|i} + v_{i|j} - v_{j|i} v_{i|j}. \tag{6}$$

In the low-dimensional embedding, UMAP defines a similar membership $w_{ij} = 1/(1 + a\|y_i - y_j\|^{2b})$ and optimizes the cross-entropy between the high-dimensional fuzzy graph v and the low-dimensional fuzzy graph w:

$$\mathcal{L} = \sum_{i,j} v_{ij} \log \frac{v_{ij}}{w_{ij}} + (1 - v_{ij}) \log \frac{1 - v_{ij}}{1 - w_{ij}}. \tag{7}$$

In practice, this is optimized with stochastic gradient descent and negative sampling. In this report, labels are used only for coloring the scatter plots, not for constructing the embedding. The procedure can be summarized as follows:



(a) Raw data **(b)** LightCNN **(c)** LSTM

(d) NM2019 **(e)** Transformer **(f)** TransformerCNN

Figure 2: UMAP visualization of raw ECG windows and the learned feature representations before the final classifier. Better separation after feature extraction indicates that the models learn discriminative temporal morphology rather than relying only on raw waveform similarity.

Algorithm 1: UMAP feature visualization
Input: raw signals or model features X; labels for coloring only.
1. Standardize or directly collect feature vectors before the final classifier.
2. Build a k-nearest-neighbor graph in the original feature space.
3. Convert neighbor distances to local fuzzy memberships using ρ_i and σ_i.
4. Symmetrize the graph by fuzzy union.
5. Initialize two-dimensional coordinates Y.
6. Optimize Y by minimizing fuzzy-graph cross-entropy.
Output: two-dimensional coordinates colored by class.

After model encoding, class clusters become more structured. NM2019 and TransformerCNN produce clearer class separation for several abnormal categories, which is consistent with their stronger classification performance. The UMAP visualization also shows why this task is not trivial: the abnormal classes are much smaller than the normal class, and some abnormal samples overlap with normal-beat morphology in the raw input space. I do not use UMAP as a quantitative metric, since it depends on the projection algorithm and hyperparameters. Instead, I use it as a qualitative representation check: a good encoder should make the feature space more class-discriminative than the original waveform space.

4.3 Class-wise Waveform and Saliency

Figure 3 shows the class-wise average ECG waveform on the test set. The five classes have visibly different average morphology, especially around the QRS complex and the post-R-peak region. This supports the choice of temporal models, because the discriminative information is not a static scalar feature but a shape pattern over time. For example, bundle branch block categories are expected to involve wider or differently shaped QRS-related patterns, while premature beats may differ in the local morphology and context around the beat. Thus, using models that preserve temporal structure is more appropriate than using hand-crafted scalar statistics alone.

Figure 4 visualizes the class-wise average temporal saliency for NM2019. The solid line is the class-wise average waveform and the dashed curve shows the normalized temporal saliency. The model does not attend uniformly across the whole 300-point window. Instead, high-saliency regions often appear around the main beat morphology and neighboring temporal regions. This behavior is reasonable for heartbeat classification: decisions should depend on QRS morphology, bundle branch block shape changes, and abnormal beat patterns rather than on unrelated flat signal regions.

At the same time, saliency should be interpreted cautiously. It is not a clinical explanation by itself, and it may vary de-

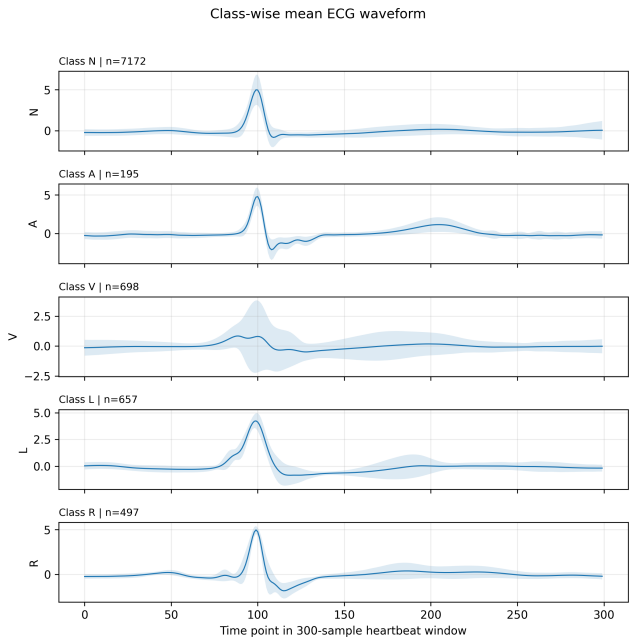


Figure 3: Class-wise average ECG waveforms on the test set. The different classes show distinct morphology around the central heartbeat region.

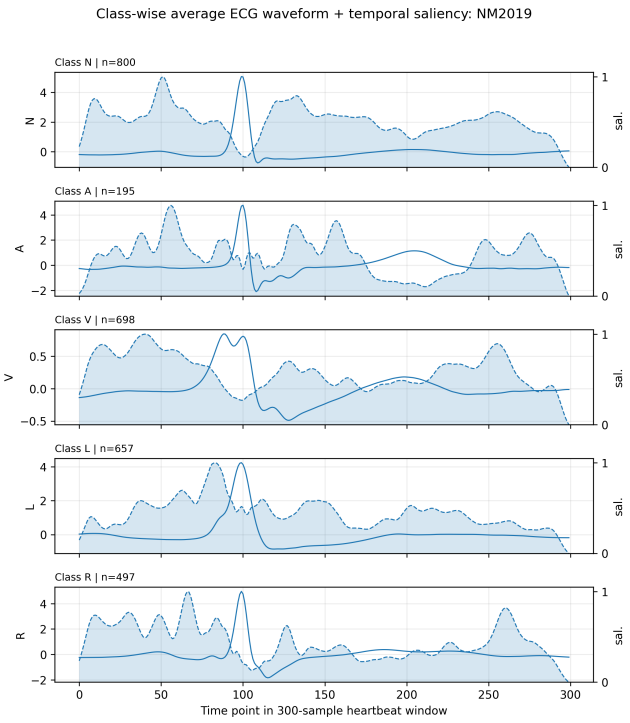


Figure 4: Class-wise average waveform and temporal saliency for NM2019. The saliency is concentrated around informative parts of the heartbeat window instead of being uniformly distributed.

pending on gradient noise, preprocessing, or model confidence. Its main value here is as a sanity check. If the highest saliency were mostly located in flat or irrelevant signal regions, it would suggest that the model might be exploiting artifacts. In this experiment, the saliency concentration around informative waveform regions makes the learned decision process more plausible.

5 Problems Encountered and Solutions

Several practical issues appeared during implementation. First, the dataset is imbalanced, with many more normal beats than abnormal beats. Therefore, besides reporting accuracy, the training code also computes balanced accuracy and F1 scores, and it supports class weighting in the cross-entropy loss. The gap between weighted-F1 and macro-F1 in Table 3 confirms that this is necessary: weighted metrics are dominated by the abundant N class, whereas macro metrics expose performance on smaller abnormal categories.

Second, different model families require different input handling. CNN-based models expect tensors in channel-first format, while the LSTM and Transformer internally convert the input into sequence or token format. I kept a consistent outer interface so that all models can be trained by the same pipeline and expose a `forward_features` function for UMAP extraction. Third, direct comparison under a single split may be misleading. I therefore repeated stratified k-fold cross validation under five seeds and reported the mean and standard deviation.

The complementary visualizations also helped check whether the models behave reasonably. Complexity analysis prevents us from selecting a model only by accuracy. UMAP reveals whether the learned representations separate classes better than raw waveform space. Saliency analysis gives a rough interpretability check by showing which temporal regions contribute most to the prediction. These checks are not replacements for test-set metrics, but they make the comparison more informative. In particular, they show that the best model is not only the one with the highest number in the accuracy table; model size, feature separability, and temporal focus should also be considered.

6 Conclusion

This assignment compared five temporal models for MIT-BIH five-class heartbeat classification. NM2019 achieved the highest accuracy, showing the strength of deep residual temporal convolution for ECG morphology modeling. TransformerCNN achieved a strong accuracy-efficiency trade-off and is a promising compact hybrid alternative. LSTM and Transformer also performed well, while LightCNN served as a useful lightweight baseline. Overall, the results show that the MIT-BIH heartbeat-window task benefits from models that can capture both local ECG morphology and broader temporal context. If only final accuracy is considered, NM2019 is the best choice in this experiment. If deployment efficiency and robustness are also considered, TransformerCNN is arguably the most attractive model among the self-implemented architectures.

References

[1] A. Y. Hannun, P. Rajpurkar, M. Haghpanahi, G. H. Tison, C. Bourn, M. P. Turakhia, and A. Y. Ng. 2019. Cardiologist-level arrhythmia detection and classification in ambulatory electrocardiograms using a deep neural network. *Nature Medicine*, 25(1):65–69.

[2] physhik. 2026. *ecg-mit-bih: Open-source implementation for ECG MIT-BIH classification*. `https://github.com/physhik/ecg-mit-bih`.

[3] G. B. Moody and R. G. Mark. 1992. *MIT-BIH Arrhythmia Database*. PhysioNet. `https://physionet.org/content/mitdb/1.0.0/`.